How A Polynomial Fit Turns Into Filter Weights

A 3x3 Example, Then the General Square, Then a Circle

1 The Core Idea

Although a polynomial fit is usually described as solving for unknown coefficients, the least-squares solution is linear in the sampled pixel values. As a result, any fitted coefficient, including a derivative coefficient, can be expressed as one fixed weighted sum of the input pixels. The resulting weighted sum for each pixel is effectively the filter weights, similar to how Sobel or Prewitt weights are defined.

2 A 3x3 Toy Example

2.1 Start With The Actual 3x3 Patch

As a simplification, let us begin with a local 3x3 image patch. We denote this neighborhood by $\boldsymbol{B}$:

$$\boldsymbol{B} = \begin{pmatrix} I_{-1,1} & I_{0,1} & I_{1,1} \\ I_{-1,0} & I_{0,0} & I_{1,0} \\ I_{-1,-1} & I_{0,-1} & I_{1,-1} \end{pmatrix}. \tag{1}$$

This matrix contains the pixel intensities in a local 3x3 window, with the pixel of interest located at the center.

Now suppose we aim to fit a degree-1 polynomial to the matrix $\boldsymbol{B}$. One simple way to represent that local polynomial is

$$p(x, y) = c_0 + c_1 x + c_2 y. \tag{2}$$

Here c_0 represents the local baseline brightness, ideally close to the brightness of the center pixel. The coefficients c_1 and c_2 describe how the brightness changes in the x and y directions, respectively. Since an edge can be understood as a spatial change in brightness, these change terms are the quantities we care about most for edge detection. In this sense, c_1 and c_2 are the derivative coefficients, while c_0 mainly helps the polynomial match the local patch.

If we instead use a degree-2 polynomial, we could write

$$p(x, y) = c_0 + c_1 x + c_2 y + c_3 x^2 + c_4 xy + c_5 y^2. \tag{3}$$

In this case, c_0, c_1, and c_2 play the same roles as before, while the additional coefficients capture curvature in the local intensity surface. Intuitively, they describe how the rate of change itself varies across the patch. Even when the polynomial degree is increased, the main coefficients used for estimating the local directional change are still the first-derivative terms. Increasing the degree simply gives the fit more flexibility, which can improve accuracy in some cases when the local image structure is not well described by a purely linear model.

With all that mathematical kerfuffle out of the way, the only question that remains is how to solve for those unknown coefficients. To do that, we write one fitting equation for each pixel in the local patch and then collect them into a single linear system.

At this point, we can write one fitting equation for each pixel in the local patch by plugging its coordinate into the degree-1 polynomial. This gives

$$c_0 - c_1 - c_2 \approx I_{-1,-1} \tag{4}$$

$$c_0 - c_2 \approx I_{0,-1} \tag{5}$$

$$c_0 + c_1 - c_2 \approx I_{1,-1} \tag{6}$$

$$c_0 - c_1 \approx I_{-1,0} \tag{7}$$

$$c_0 \approx I_{0,0} \tag{8}$$

$$c_0 + c_1 \approx I_{1,0} \tag{9}$$

$$c_0 - c_1 + c_2 \approx I_{-1,1} \tag{10}$$

$$c_0 + c_2 \approx I_{0,1} \tag{11}$$

$$c_0 + c_1 + c_2 \approx I_{1,1}. \tag{12}$$

These are simply the nine pixel-fitting equations written one at a time. The reason we stack the patch into a vector is that it allows us to collect these nine equations into one compact linear system of the form

$$\boldsymbol{A}\boldsymbol{z} \approx \boldsymbol{b}. \tag{13}$$

2.2 Stack The Patch Into A Vector

To write that system compactly, we flatten those same nine pixel values into a column vector

$$\boldsymbol{b} = \left[I_{-1,-1}, I_{0,-1}, I_{1,-1}, I_{-1,0}, I_{0,0}, I_{1,0}, I_{-1,1}, I_{0,1}, I_{1,1}\right]^\top. \tag{14}$$

The unknown polynomial coefficients are gathered into

$$\boldsymbol{z} = [c_0, c_1, c_2]^\top. \tag{15}$$

2.3 Build The Design Matrix

Each row of the design matrix is just the basis $[1, x, y]$ evaluated at one sample location. Therefore

$$\boldsymbol{A} = \begin{pmatrix} 1 & -1 & -1 \\ 1 & 0 & -1 \\ 1 & 1 & -1 \\ 1 & -1 & 0 \\ 1 & 0 & 0 \\ 1 & 1 & 0 \\ 1 & -1 & 1 \\ 1 & 0 & 1 \\ 1 & 1 & 1 \end{pmatrix}. \tag{16}$$

2.4 Solve The Least-Squares System

The fitted coefficients are

$$\hat{\boldsymbol{z}} = (\boldsymbol{A}^\top \boldsymbol{A})^{-1} \boldsymbol{A}^\top \boldsymbol{b}. \tag{17}$$

For this particular 3x3 geometry, one can compute

$$\boldsymbol{A}^\top \boldsymbol{A} = \begin{pmatrix} 9 & 0 & 0 \\ 0 & 6 & 0 \\ 0 & 0 & 6 \end{pmatrix}, \tag{18}$$

so

$$(\boldsymbol{A}^\top \boldsymbol{A})^{-1} = \begin{pmatrix} \frac{1}{9} & 0 & 0 \\ 0 & \frac{1}{6} & 0 \\ 0 & 0 & \frac{1}{6} \end{pmatrix}. \tag{19}$$

Thus the pseudoinverse is

$$\boldsymbol{P} = (\boldsymbol{A}^\top \boldsymbol{A})^{-1} \boldsymbol{A}^\top. \tag{20}$$

The second row of $\boldsymbol{P}$ gives the weights for c_1, the x-derivative coefficient:

$$\boldsymbol{p}_{c_1}^\top = \left[-\frac{1}{6}, 0, \frac{1}{6}, -\frac{1}{6}, 0, \frac{1}{6}, -\frac{1}{6}, 0, \frac{1}{6}\right]. \tag{21}$$

So the fitted derivative coefficient is

$$\hat{c_1} = \boldsymbol{p}_{c_1}^\top \boldsymbol{b}. \tag{22}$$

Written out explicitly,

$$\hat{c_1} = \left(-\frac{1}{6}\right) I_{-1,-1} + 0 I_{0,-1} + \left(\frac{1}{6}\right) I_{1,-1} \tag{23}$$

$$+\left(-\frac{1}{6}\right) I_{-1,0} + 0 I_{0,0} + \left(\frac{1}{6}\right) I_{1,0} \tag{24}$$

$$+\left(-\frac{1}{6}\right) I_{-1,1} + 0 I_{0,1} + \left(\frac{1}{6}\right) I_{1,1}. \tag{25}$$

This is the crucial moment. The polynomial fit has turned into a 3x3 derivative filter:

$$\boldsymbol{K}_x = \begin{pmatrix} -\frac{1}{6} & 0 & \frac{1}{6} \\ -\frac{1}{6} & 0 & \frac{1}{6} \\ -\frac{1}{6} & 0 & \frac{1}{6} \end{pmatrix}. \tag{26}$$

The fit sounds like solving for a polynomial, but because the fit is linear, the derivative we extract is just one fixed weighted sum of the nine pixels.

3 Generalization To An N x N Square

Now suppose the support is an arbitrary square window with coordinates

$$(x_i, y_i), \quad i = 1, 2, ..., N_p, \tag{27}$$

where for a square window $N_p = N^2$.

For a polynomial of degree d, define the monomial basis vector

$$\varphi_{d(x,y)} = \left[1, x, y, \frac{x^2}{2}, xy, \frac{y^2}{2}, ...\right]^\top, \tag{28}$$

with total length

$$M = (d+1)\frac{d+2}{2}. \tag{29}$$

Then the design matrix is built exactly the same way:

$$\boldsymbol{A} = \begin{pmatrix} \varphi_{d(x_1,y_1)}^{\top} \\ \varphi_{d(x_2,y_2)}^{\top} \\ \vdots \\ \varphi_{d\left(x_{N_p},y_{N_p}\right)}^{\top} \end{pmatrix}. \tag{30}$$

The fitted coefficients are still

$$\hat{\boldsymbol{z}} = \left(\boldsymbol{A}^{\top}\boldsymbol{A}\right)^{-1}\boldsymbol{A}^{\top}\boldsymbol{b} = \boldsymbol{P}\boldsymbol{b}. \tag{31}$$

If the derivative coefficient we want is the entry corresponding to x, then one row of $\boldsymbol{P}$ gives a weight vector

$$\boldsymbol{p}_{f_x}^{\top}, \tag{32}$$

and the derivative estimate is

$$\hat{f}_x = \boldsymbol{p}_{f_x}^{\top}\boldsymbol{b}. \tag{33}$$

So nothing changes conceptually from 3x3 to NxN. The matrix gets larger, but the derivative is still one row of the pseudoinverse dotted with the pixel vector.

4 From A Square To A Circle

The circular case is not a new algorithm. It is the same least-squares construction with a different set of sample locations.

Instead of using every point in a square, keep only the points satisfying

$$x_i^2 + y_i^2 \leq r^2. \tag{34}$$

Now the data vector contains only the pixel values inside that circular support,

$$\boldsymbol{b} = \left[I_1, I_2, ..., I_{N_p}\right]^{\top}, \tag{35}$$

and the design matrix keeps only the corresponding rows

$$\boldsymbol{A} = \begin{pmatrix} \varphi_{d(x_1,y_1)}^{\top} \\ \varphi_{d(x_2,y_2)}^{\top} \\ \vdots \\ \varphi_{d\left(x_{N_p},y_{N_p}\right)}^{\top} \end{pmatrix}, \tag{36}$$

where the coordinates now come from the circular neighborhood rather than the full square.

The least-squares solution is unchanged:

$$\hat{\boldsymbol{z}} = \left(\boldsymbol{A}^{\top}\boldsymbol{A}\right)^{-1}\boldsymbol{A}^{\top}\boldsymbol{b}. \tag{37}$$

And again, the derivative estimate is one row of the pseudoinverse:

$$\hat{f}_x = \boldsymbol{p}_{f_x}^{\top}\boldsymbol{b}. \tag{38}$$

So the circle does not change the logic at all. It only changes which sample coordinates appear as rows of $\boldsymbol{A}$.

5 Why This Matters For WVF

The WVF does two additional things.

First, it rotates the local coordinates so that the derivative is taken in the candidate normal direction:

$$x_{i'} = \Delta x_i \cos\theta + \Delta y_i \sin\theta, \tag{39}$$

$$y_{i'} = -\Delta x_i \sin\theta + \Delta y_i \cos\theta. \tag{40}$$

Second, it uses a circular support instead of a square one.

But once those rotated circular sample locations are fixed, the exact same least-squares logic applies:

$$\hat{\boldsymbol{z}} = \boldsymbol{P}_\theta \boldsymbol{b}, \tag{41}$$

and the WVF weight vector is just the row of $\boldsymbol{P}_\theta$ corresponding to the normal derivative coefficient:

$$\hat{f}_{x'} = \boldsymbol{p}_{f_{x'}}^\top \boldsymbol{b}. \tag{42}$$

That row is the WVF stencil.

6 The Short Version

The shortest correct explanation is this.

The polynomial fit is a linear map from sampled pixel values to polynomial coefficients. Because the derivative coefficient is one entry of the fitted coefficient vector, it is also a linear map of the sampled pixels. Any linear map from pixels to a scalar can be written as one fixed weighted sum. Those fixed numbers are the filter weights.

So the WVF weights are not something added after the polynomial fit. They are exactly the polynomial fit, collapsed into the one row needed to extract the derivative.